

Computer Graphics: Lecture 7

Textures and Bind Groups

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](#)

Welcome Back!

Last time we learned all about color blending and vertex attributes.

We learned that every vertex can have multiple attributes attached to it, and that this data is passed to the vertex shader, one per `@location`.

We learned that we can interleave attributes by putting each attribute in some order, one per vertex.

We learned that the stride of a vertex buffer is how much to add to get from one vertex to another. This is the size of a vertex if attributes are interleaved. If we split attributes into different buffers, this will be the length of one “row”.

This time

We're finally going to learn about textures.

This is an important topic. Textures are the cornerstone of 3D graphics. Not only do they give objects their detail, they can be used as lookup tables for all kinds of effects (shadows, normal maps, reflections, the sky, many more things)

Furthermore, when we render a frame, we're actually drawing into a texture (which is why I've been handwaving `context.getCurrentTexture()`).

Textures are a little tough, so this lecture may take **two** class periods.

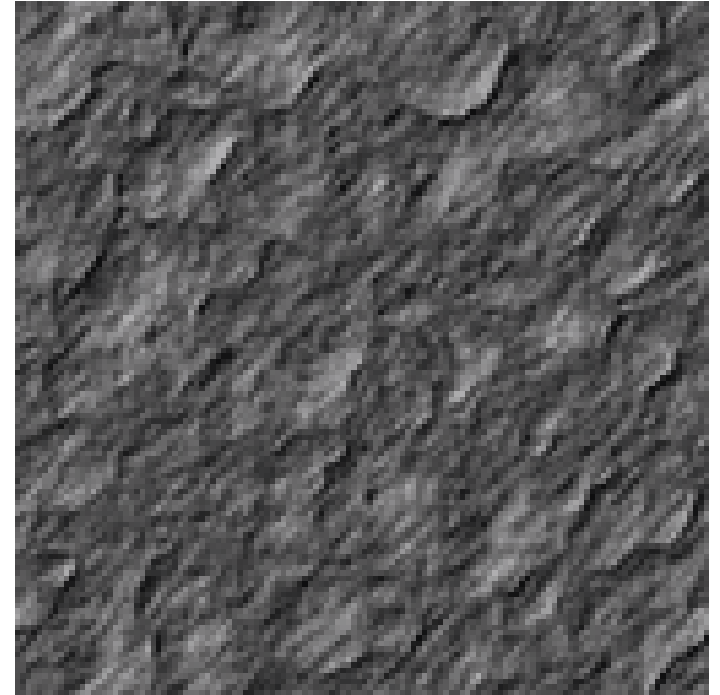
What is a texture?

Textures are images that we project on 3D shapes

But this is a bit oversimplified.

In fact, they have a complex internal storage, with multiple levels of images of different sizes.

We will learn the details in the advanced texturing later in the course.



A texture. (by [GGBot](#)—scaled up beyond its intended dimensions)

Why textures?

Textures actually serve a lot of purposes.

Our first use of them will be to give detail to 3D objects.

Since we haven't learned about perspective yet (it's coming, I promise!), textures will be a way to draw pixels on the screen instead of just colored gradients.

They can also be lookup tables, temporary storage, palettes, depth buffers, and even define shadows. They are extremely versatile.

What textures?

Textures can be 1-dimensional, 2-dimensional, or 3-dimensional. We will discuss 2-dimensional textures mainly in this course as those are the most commonly used.¹

In the same way that a normal image is composed of *pixels*, textures are composed of **texels**.

A **texel** is a *TEXTure ELement*.

Why a different term? Because like I mentioned before, textures actually have multiple levels. A texel is a pixel in one of the levels.

¹1D textures are useful for gradients and borders, 3D textures are useful for volumetric fog and medical imaging.

How textures?

In 3D textures are not just images we draw “at an angle”.

Instead, we draw a triangle, but we **sample** the texture as we do so.

Sampling a texture is like looking up a pixel in an image, but it is smooth. We can smoothly look up a point in between several texels.

We do this for every fragment on the triangle.

The key is knowing specifically which texels to look up at which point on the triangle.

Let's get concrete

Textures are a little weird, but once you understand how they work, they gradually become intuitive.

To make them intuitive, we're going to focus on one simple problem: drawing a textured quad.

That is, we're going to draw a rectangle, but we'll modify our fragment shader to look up the texels instead of blending a color.

UV coordinates

Textures are sampled using a special coordinate system. Understanding this coordinate system is critical to getting good at textures.

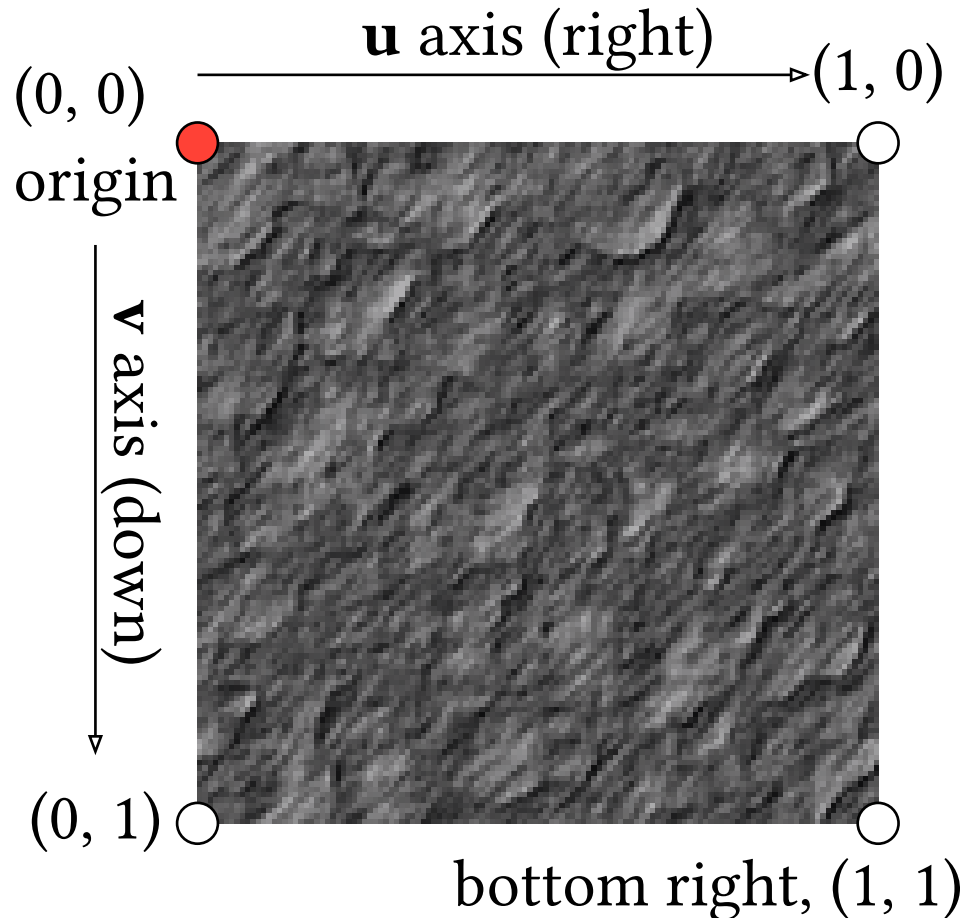
The coordinate system is called **uv** coordinates.

u and **v** are dimensions, like xy or rgb.

They are a way to describe the position of a sample on a texture, regardless of how big the texture is.

Let's see how they work...

Example UVs



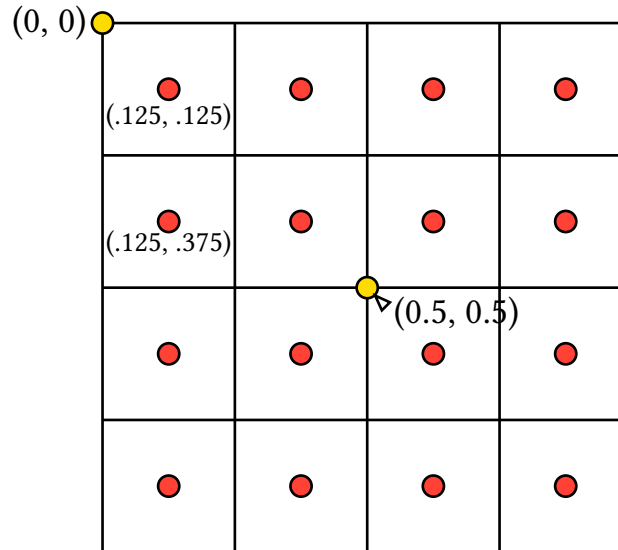
The u, v coordinate system treats the *origin* as the top left.

“ u ” and “ v ” refer to far to go down and right.

Our goal is to choose a point to “sample”, meaning, to retrieve the color at.

If we choose $(0.5, 0.5)$, we are sampling the very center of the image.

Here is a 4-by-4 texture with UV coordinates



Each cell is .25 wide and tall. The center (0.5, 0.5) is between 4 texels. If we sample that point, the GPU could give us one of those 4 colors, or it could average them together depending on our sampling mode.

With me so far?

Textures are like images, with *texels* instead of *pixels*.

The main things we do to textures is *sample* them, which means to look up a color from a position on the texture.

We don't provide the raw coordinates of the texel, instead we use the u/v coordinate system.

We do this so that we could replace the texture with a higher-resolution one (more pixels) and have the addressing still return roughly the same colors.

With me so far? (2)

U/V coordinates have $(0, 0)$ at the top left. The U coordinate points right, the V coordinate points down.

$(0.5, 0.5)$ is the *center* of the texture. For a power of 2 sized texture it refers to the center of a square of 4 texels.

$(0, 0)$ refers to the top left corner of the top left texel.

To get the actual center of the texel, you have to add an offset, which is half the width of a texel in u/v coordinates.

$1/\text{width}$ is the width of a texel in u coordinates. Half of that is the distance from the left border of a texel to its center. Likewise for V.

Questions?

Texture mapping a triangle

The act of applying a texture to something is called *texture mapping*.

Many texturing techniques have “mapping” in their name: normal mapping, shadow mapping, parallax mapping, etc.

What we want to do is color mapping. Have the *color* of a fragment come from a texture, instead of being blended from vertex output colors like we did last time.

Texture mapping a triangle (2)

Previously, we took advantage of the fact that colors were *interpolated* before the hit the fragment shader.

That is, if one vertex of a triangle was red, one was green, and one was blue, the fragment shader would receive the correct blend of red, green, and blue depending on how close it was to each vertex.

We want to do *the same thing* but with u/v coordinates instead of colors.

If we do that, we'll end up “blending” the u/v coordinates, so that nearby fragments sample nearby texels.

Texture mapping example

To understand this, let's imagine a single triangle.

This triangle will have a u/v attribute, which stores the u/v coordinates at each of its three vertices.

Each fragment on the surface of the triangle will interpolate between these three u/v values.

Inside the fragment shader, when the fragment samples the texture, it will pull a color that corresponds to the unique u/v coordinates of that fragment.

Texture mapping example (2)

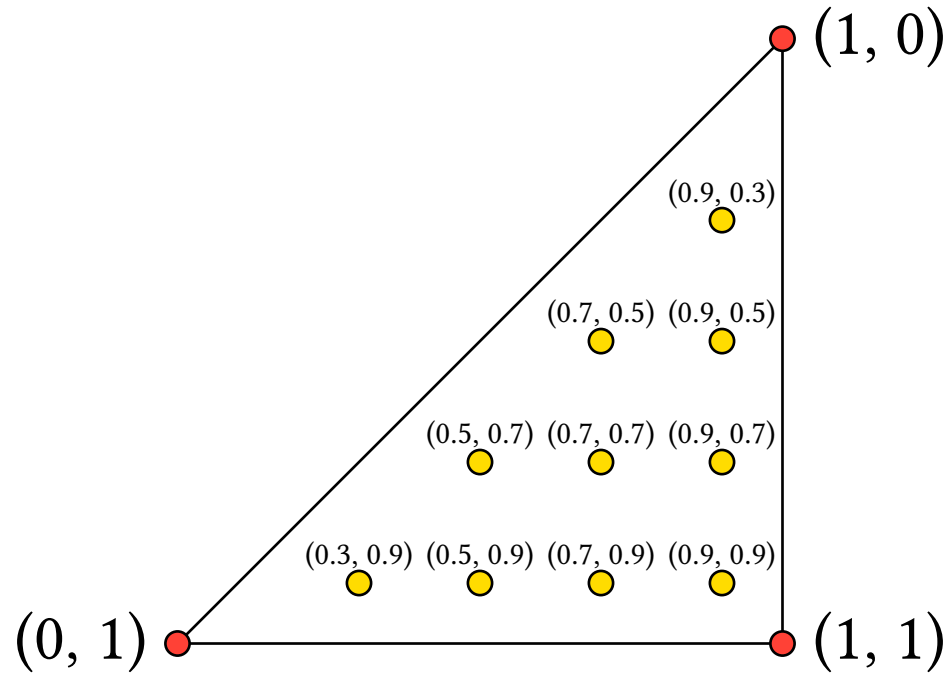
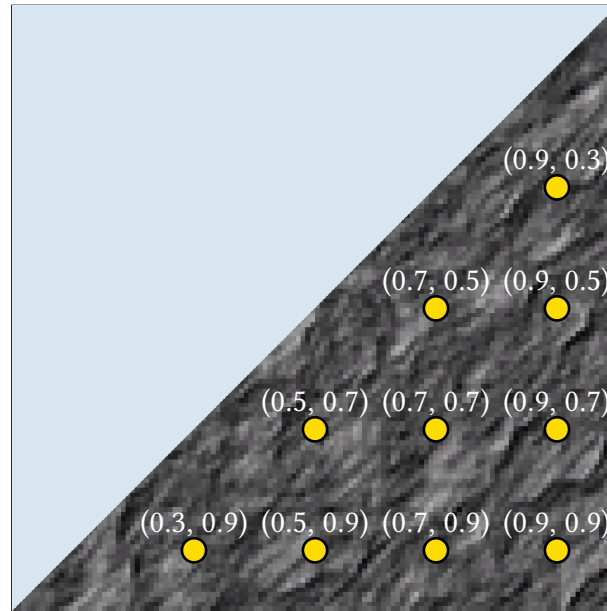


Figure 2: Each fragment (pixel within triangle) will receive an interpolated u/v coordinate. Therefore, the color looked up from the texture will be different for each one.

Texture mapping example (3)



Each fragment gets a uv coordinate that is interpolated between the uv coordinates of the corners. This happens for *every fragment*, but I'm only showing a few. (Imagine that there is a dot for every pixel on the screen)

Comprehension check

What would happen visually if we continued to sample $(1, 1)$ for the bottom left point, but we sampled $(1, .75)$ for the top right point, and $(.75, 1)$ for the bottom left point, but the triangle is the same size.

What would it look like?

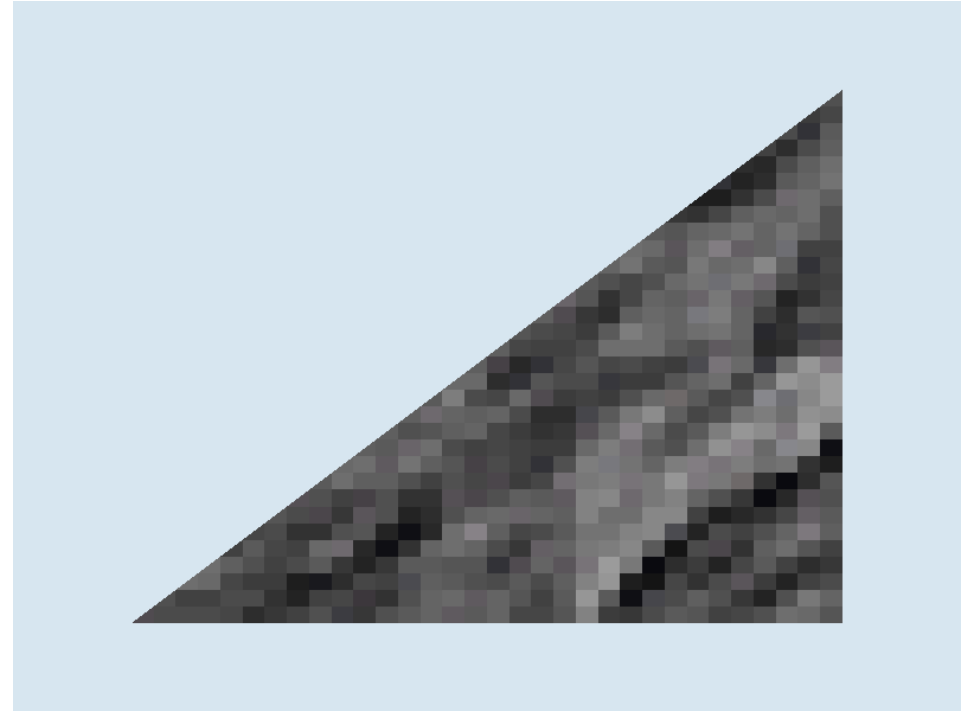
[Take a second and think about it]...

It would zoom in on the corner

We sample a smaller portion of the texture (the lower right sixteenth of it), but we are blowing it up over the same size of rectangle.

The number of pixels we're drawing stays the same, but the number of *texels* we're sampling is 1/16th as many.

Therefore, the texture is zoomed in, and looks grainy.¹



¹this is why games from the 5th generation of game consoles (i.e., the PS1, N64 generation) look so grainy/blurry. The textures had to be small due to hardware limitations.

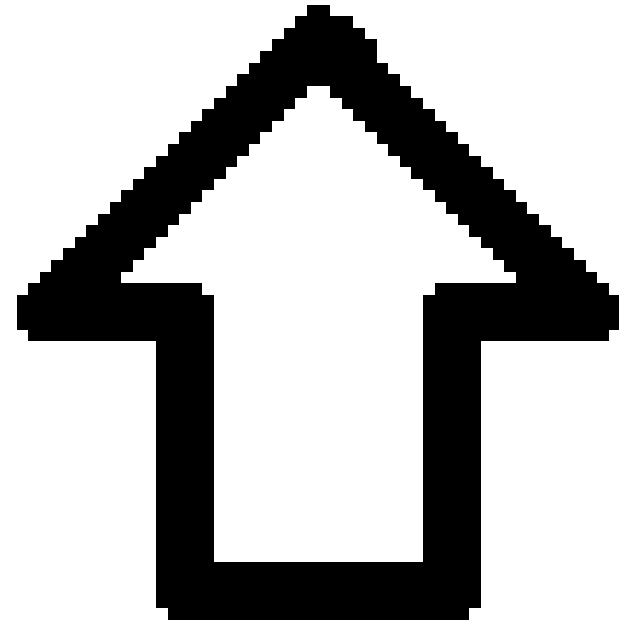
Questions?

Let's get artistic

Now that we understand a bit more about UV coordinates, let's try to think artistically about them.

That is, instead of imaging *what* it will look like, let's try imagining how to achieve an effect that we want. This is a question we will often find ourselves asking.

Suppose we have a texture that looks like an up arrow.



This is a 64 by 64 texture, so we can see the texels.

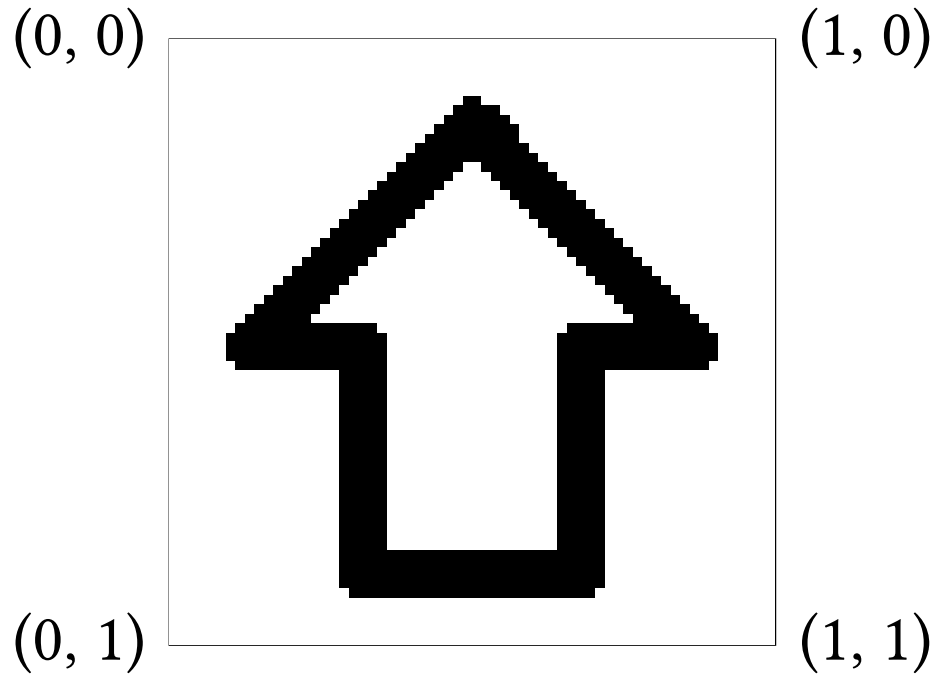
Our first goal

Let's just draw the arrow, but instead of a triangle, let's draw it to a quad, instead.

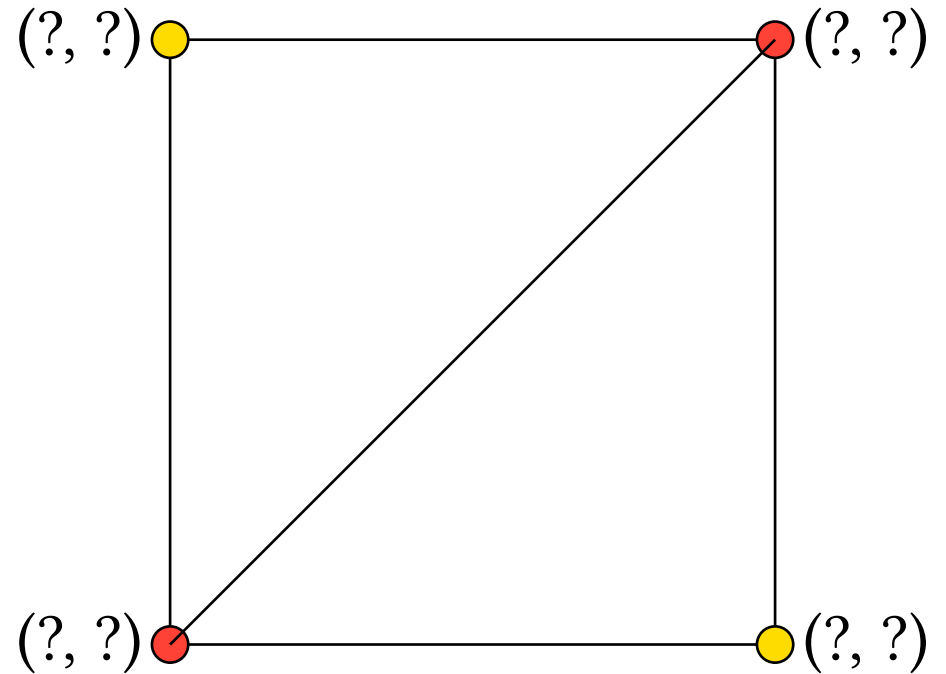
That means we will need to specify 6 vertices¹, two triangles.

¹There's a way to specify a quad with 4 verts using a different topology, but we haven't covered those yet.

What are the U/V coordinates for each vertex?

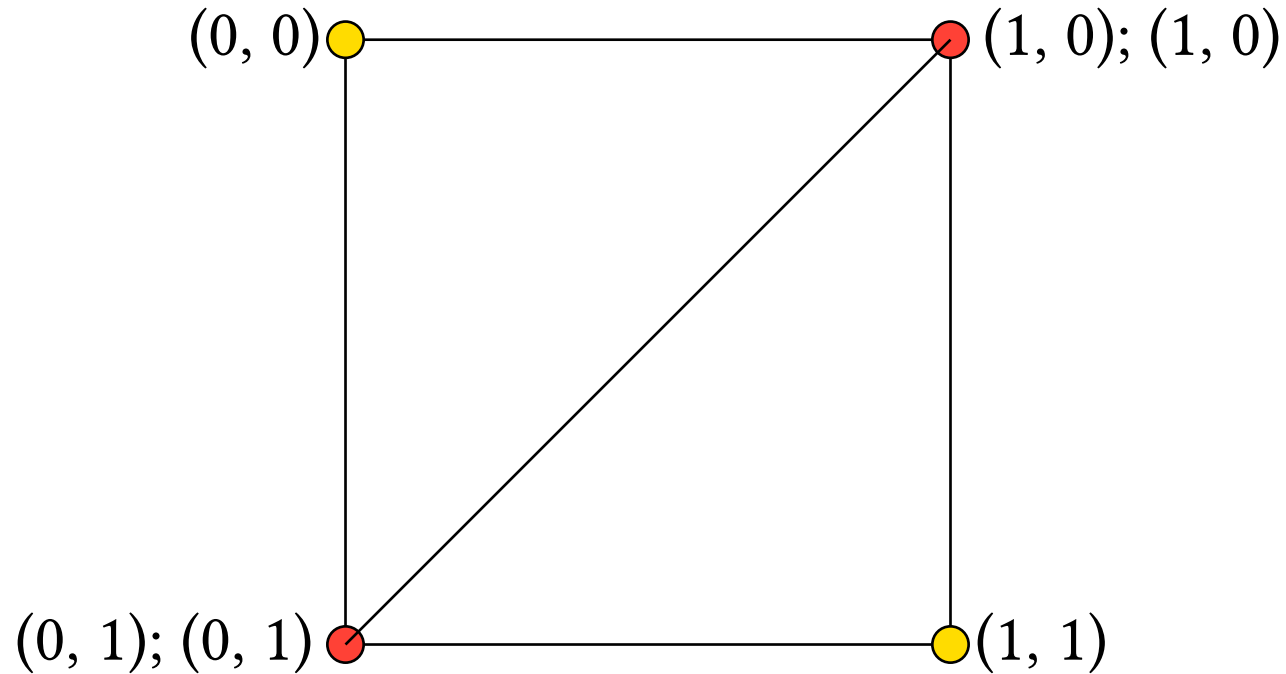


The texture, with the corner UV coordinates labelled. These are always the same for every texture.



The top right and bottom left points are duplicated between both triangles, but we want them to have the same U/V coordinates (otherwise there'd be a seam)

U/V coordinate answer



It's the same. We map the top left of the texture to the top left of the quad. We map the top right of the texture to both vertices in the top right. Ditto for bottom left. We map bottom-right to bottom-right.^{23 / 66}

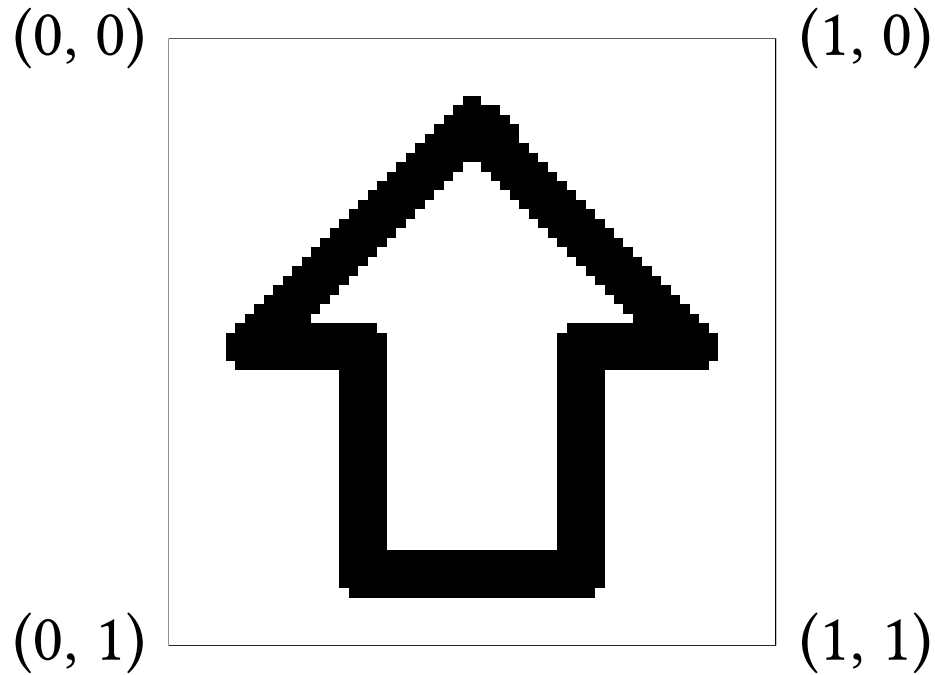
The principle

The basic idea when answering this question is: “what part of the texture do I want to be at this point on the triangle.”

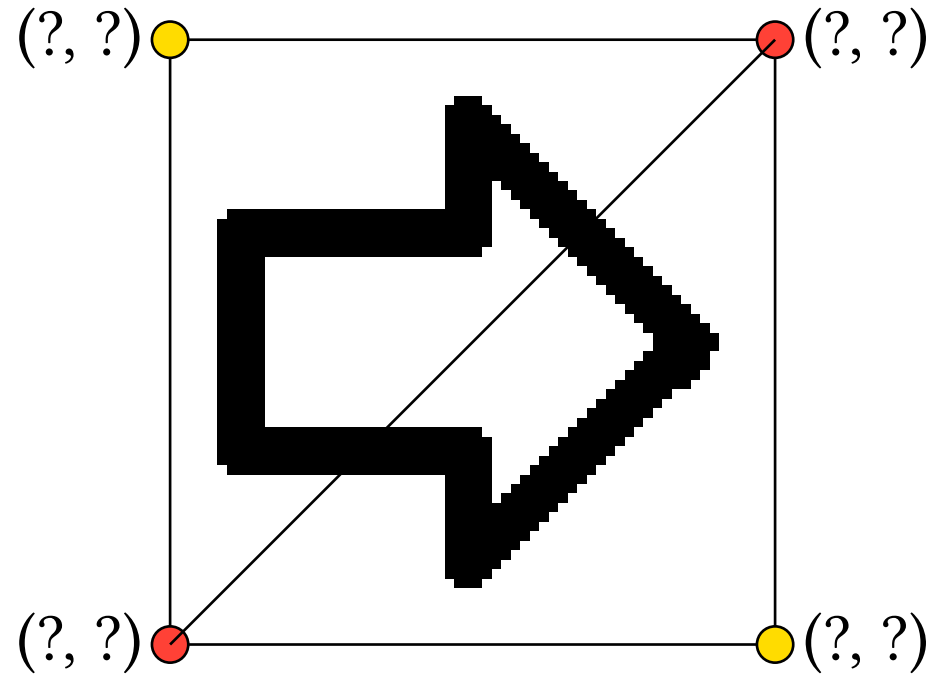
So let’s make it a bit harder. How do we map the triangle *sideways*?

Specifically, we want the arrow to be pointing to the right. This means we need a new batch of U/V coordinates. Which ones?

Right arrow mapping question?

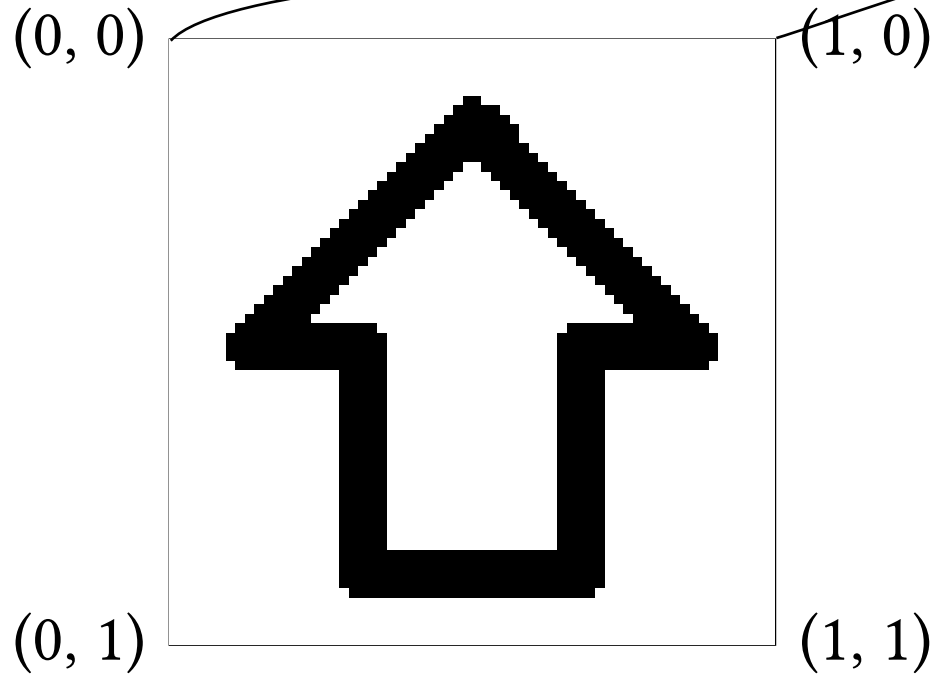


The texture, with the corner UV coordinates labelled. These are always the same for every texture.

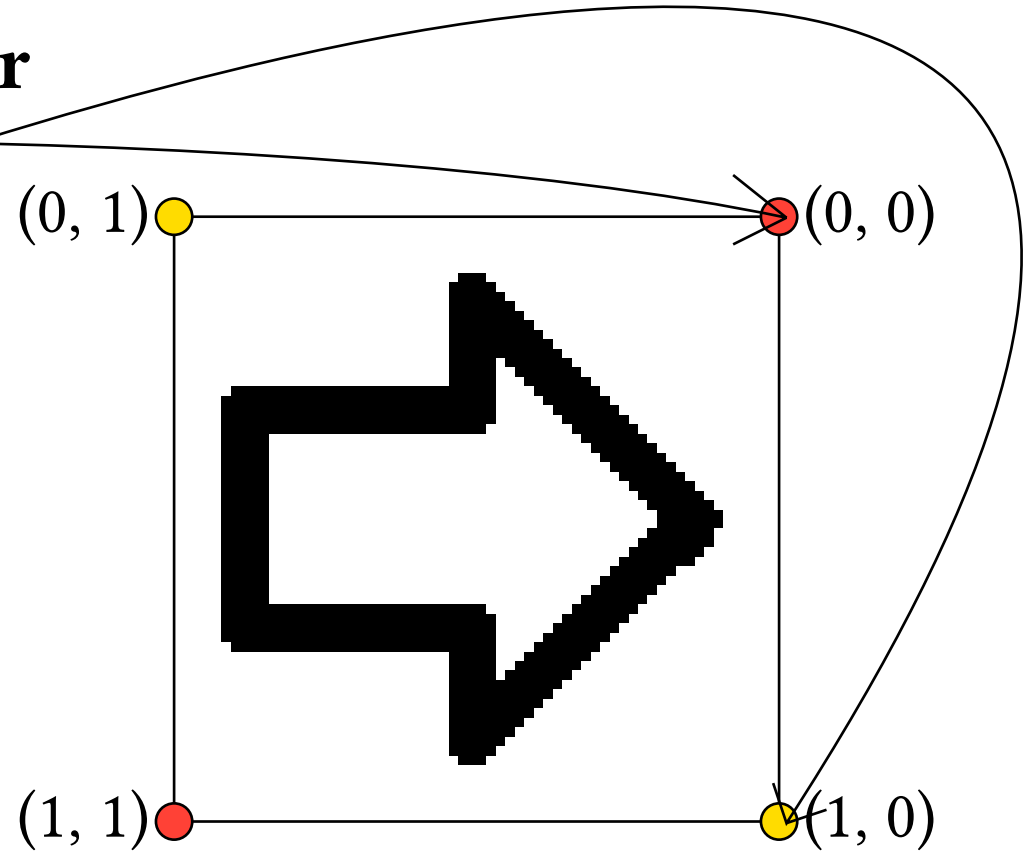


Now we want the arrow to be pointing right.

Right arrow mapping answer



Texture U/V coordinates are always the same. What changes is the U/V coordinates we use in the model, which can rotate, shift, or otherwise warp the texture.



We make the top right U/V coordinate $(0, 0)$ so that it pulls from the top left of the texture. We make the bottom right $(1, 0)$, and so on for the other two..

How are we doing?

Can we see how this system allows us the flexibility to map the texture however we want?

The idea is that at each vertex, we're choosing what part of the texture we're pulling from.

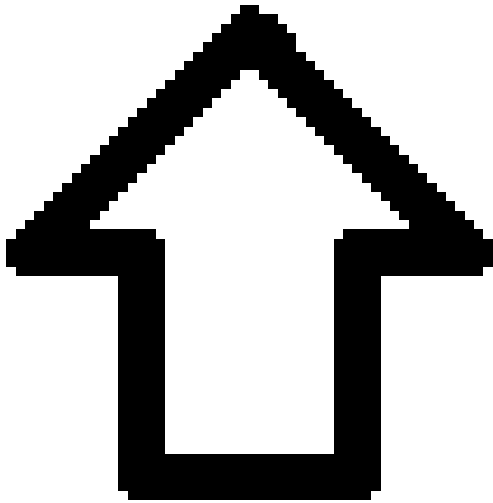
The GPU will interpolate between the U/V coordinates of the vertices.

Think of the triangle as a cookie-cutter that can be stretched, moved, rotated, or sheared however you want. Think of the texture as the cookie dough.

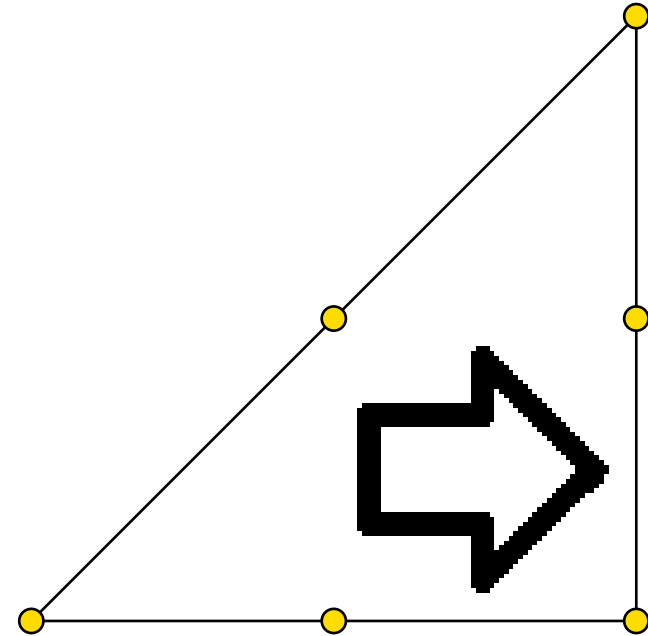
Feel free to ask questions. Then, let's do one more...

Last UV mapping question

Now let's suppose we want to map this arrow so that it is facing right, and centered within our right triangle. We can add vertices if we want.



What we have



What we want

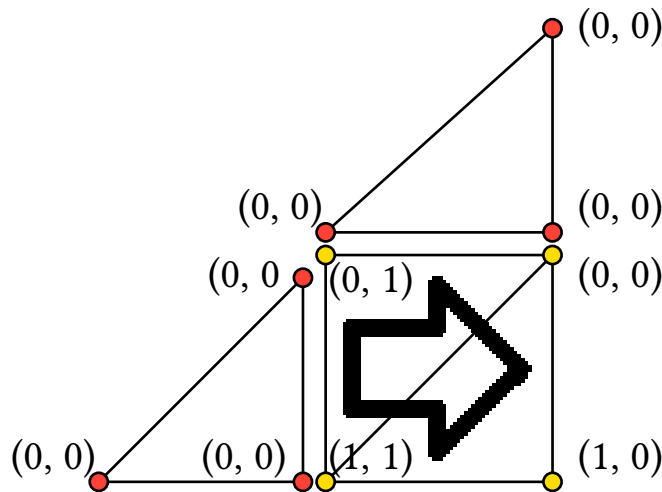
Any ideas?

Some additional things to keep in mind:

- Not every point has to sample the texture. We can use special U/V coordinates that says “just make this vertex blank”
- If there is a triangle with all the same U/V coordinates, it will have a uniform color. We can make sure that color will be white.
- Two vertices can be in the same location but have different U/V coordinates. Each vertex gets to have its own attribute values.

One solution

One way to do this is to break up (tessellate) our triangle:



Some points are duplicates. I drew the triangle broken, but imagine that the points of the square in the middle share the same position as the north and west triangles' edges. The hypotenuse midpoint position is repeated 3 times, but for the inner square it has $(0, 1)$ U/V coordinates.

Questions?

Let's code it up

So how do we actually code a texture-mapped quad?

Let's start with the new shaders.

The vertex shader is similar, only now it takes a uv coordinate vector instead of a color.

The fragment shader is where it gets different...

Let's start with the more familiar part, then show the part where we add textures.

The vertex part

The vertex shader is similar to what we had before, only now instead of a 3-vector color, we take a 2-vector of u/v coordinates:

```
struct VertexOutput {  
    @builtin(position) pos: vec4f,  
    @location(0) uvs: vec2f,  
};  
  
@vertex fn vs(  
    @location(0) position: vec2f,  
    @location(1) uvs: vec2f,  
) -> VertexOutput {  
    var vo: VertexOutput;  
    vo.pos = vec4f(position, 0, 1);  
    vo.uvs = uvs;  
    return vo;  
}
```

The bind groups and fragment shader

```
@group(0) @binding(0) var tex: texture_2d<f32>;  
@group(0) @binding(1) var samp: sampler;  
@fragment fn fs(vo: VertexOutput) -> @location(0) vec4f {  
    return textureSample(tex, samp, vo.uvs);  
}
```

This is the new part.

We store a texture and a sampler, using this new @group(. . .) @binding(. . .) notation. I haven't even mentioned samplers yet, don't worry!

Then, for the color, we return the color we textureSample.

Three new things

So there are three new things to cover:

- There's a texture type. `texture_2d<f32>`. `<f32>` means that the texels will be sampled as floats, which is almost always what we want.
- `@group` refers to something called a bind group. We need to cover that.
- Samplers are important, too.

Bind groups are important for more than just textures: they show up whenever there is data that is needed for drawing an entire 3D model.

They are like vertex buffers, but instead of their data being different for each vertex, it's the same for the entire draw call. This is useful for all kinds of things...

Bind groups utility

For example, let's say we're drawing a tree in a game or sim.

We want to give each tree its own texture to make them look unique, but maybe they have the same 3D mesh (triangle data).

Each time we draw the tree, we switch to the new texture first.

Textures (and samplers) are stored in something called a **bind group**.

We create a **bind group**, at least one for each object (but often several), and load the per-object data into it.

In the shader, we declare variables that come from the bind group.

Bind groups utility (2)

The main purpose of bind groups is to only allow us to change the things that we need to, and leave everything else alone.

For example, if lots of objects have different positions but the same texture, we can put those pieces of data in different bind groups, and only swap out the ones we want.

Typically, an object will have at least one bind group for that object (its position, texture, things like that), one bind group for the entire pass (shadow maps, some shared textures), and one bind group for the entire draw (global information like time).

Bind group layout

To create a bind group, we need a **bind group layout**. This describes what kinds of variables are stored in the bind group and what their binding numbers are.

Defining a layout is very similar to defining attributes.

Once we define a layout, we can use it to create bind groups and load them with the data we want the object to have when we're drawing it.

Then, we call `pass.setBindGroup(number, bindgroup)` to actually have this data get sent to our shader.

Samplers

There is one more thing we need to cover before we continue: sampling.

We never use a texture by itself. We always pair them with another object called a **sampler**.

A sampler stores information about *how* a texture is sampled. It answers several questions, such as:

- What happens if you sample a point that isn't exactly in the center of a texel. How should we compute the color?
- What happens if you sample a point *outside* the texture? Believe it or not, this can be very useful.
- What happens if the texture is really small? (surprisingly important)

Samplers (2)

It will be easier to understand samplers if we see them in action.

Therefore, let's finish the code for the texture sample first, so we can start to experiment.

First, let's create a texture.

Then, let's create a sampler.

Finally, let's create bind groups and load them with the texture and sampler that we want for our quad.

Creating a texture

There are basically two ways to create a texture:

1. Loading an image from a file
2. Creating one procedurally

We will cover both.

Procedural generation means generating something with a procedure.

The term is often associated with random generation, but the thing you make doesn't have to be random (and won't be for us)

We are going to procedurally generate **the XOR texture**.

Texture storage

Textures can be stored in many different formats.

The most common are `rgba8unorm` and `rgba8unorm-srgb`.

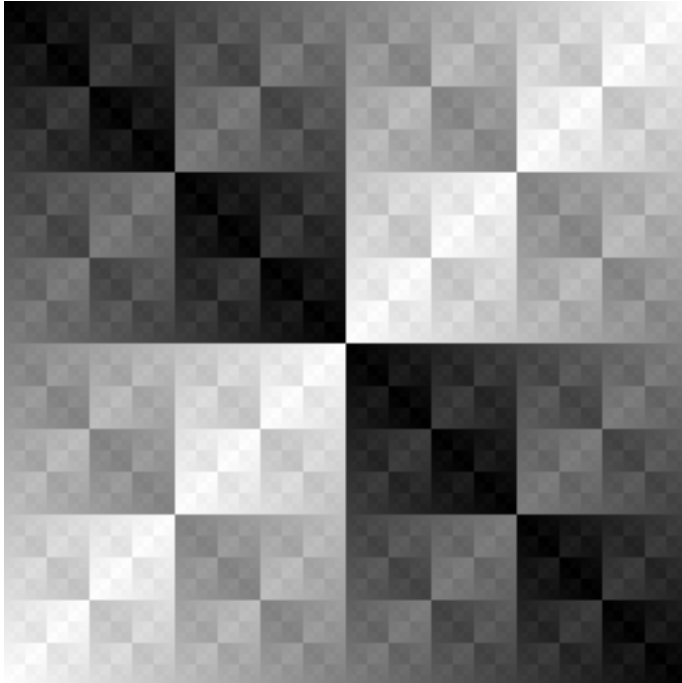
The way this format works is that each pixel is 32 bits: 8 bits for red, 8 bits for green, 8 bits for blue, and 8 bits for *alpha*.

The `u` means unsigned, and the `norm` means normalized to $[0, 1]$.

The numbers are unsigned integers, and are interpreted as being in the range $[0, 1]$, where 0 is 0, and 1 is 255.

We'll talk about *alpha* later. It basically represents the *amount* of color. For now it will be 100%.

The XOR texture



The XOR texture is created by making the brightness of each texel just be the XOR between the row and column number of that texel.

So the texel at position 67, 100 has brightness $67 \oplus 100 = 39$.

We set this value to the value of R, G, and B. A is set to 255. So (39, 39, 39, 255) is the color.

Creating textures

In our sample, you can see how I define the xor texture texels, but we also need to allocate space for the texture on the GPU:

```
const tex = device.createTexture({  
  format: "rgba8unorm-srgb",  
  size: {width, height, depthOrArrayLayers: 1},  
  usage:  
    GPUTextureUsage.TEXTURE_BINDING |  
    GPUTextureUsage.RENDER_ATTACHMENT |  
    GPUTextureUsage.COPY_DST,  
  dimension: '2d',  
  label: "xor texture"  
});
```

Creating textures (2)

The format is as I said before: we use `rgba8unorm`. We add the `-srgb` to inform the GPU that we intend the color values in the texture to be *perceived brightness* (e.g., gamma correct).

When we sample the texture, the GPU will convert these values to *linear brightness* by raising them to the correct power.

After format, we specify size. It has width and height fields in pixels, but also the number of images. 2D texture data can be stored in texture arrays, which is what `depth0rArrayLayers` is for (along with 3D texes).

We're working with 2d textures, so we specify the dimension

Creating textures (3)

The usage field needs some explanation. There are 3 bitflags that we or together in order to define how we will use the texture.

This is what the specific values mean:

- `TEXTURE_BINDING` means that it will be bound as a texture. Obviously we want that one.
- `COPY_DST` means that we can copy data into it. We need this flag because we are about to do that (this is how the data gets loaded into the texture).
- `RENDER_ATTACHMENT` means “it will be written to in a render pass”. This happens when we load the texture (soon).

Loading textures

What we just did was basically allocate the texture. Now let's load it up.

First, we define an `ImageData` object, which stores the pixels from our XOR texture (see the sample for how we generated it):

```
const texData = new ImageData(  
    buf, width, height,  
    {colorSpace: "srgb", pixelFormat: "rgba-unorm8"}  
);
```

We have to make sure to tell it that the pixels are already perceived brightness (srgb), otherwise, it would convert it when loading the texture (because we said the texture was expected to be srgb).

Loading textures (2)

Now, we actually copy the bits over. This is done using the queue:

```
device.queue.copyExternalImageToTexture(  
    {source: texData},  
    {texture: tex, colorSpace: "srgb"},  
    {width, height, depthOrArrayLayers: 1}  
);
```

Here we say “this is the texture data I want to copy, here is the destination texture and I expect it to be srgb, and finally, here’s how *much* to copy”.

What about from a file?

To load a texture from a file, we need *asynchronous* code.

The reason is that loading files takes a long time, so it's better if we do it in parallel. We can start loading a texture, then when we need it later, check if it's done.

Here's how we define an async function. It's like a normal function except it returns a `Promise<...>`, which is a value that represents something that will be available in the future:

```
export async function loadTexture(  
  device: GPUDevice,  
  url: URL,  
): Promise<GPUTexture> { ... }
```

Loading a texture from a file

The next lines of the function:

```
const response = await fetch(url);  
const blob = await response.blob();  
const bitmap = await createImageBitmap(blob);
```

`await` means “I want to pause until the promise is finished”

`fetch` returns a promise of a response, which is an HTTP response. It stores a status code and a progress state (i.e., how much is loaded).

We await the `blob`, which is the raw data in the packet.

Then, we use that data to create a bitmap

Loading a texture from a file (2)

Once we have the bitmap, we load it the same way we did before:

```
const tex = device.createTexture(...);  
  
device.queue.copyExternalImageToTexture(  
    {source: bitmap},  
    {texture: tex, colorSpace: "srgb"},  
    { width: bitmap.width,  
      height: bitmap.height,  
      depthOrArrayLayers: 1 }  
);
```

Creating the sampler

Now that we have a texture, we need to define a sampler.

A sampler tells the GPU *how* we want to sample the texture.

Let's create one, and I'll explain the options:

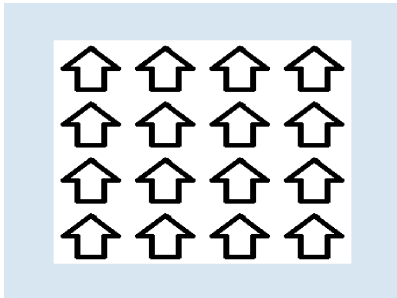
```
const samp = device.createSampler({  
  addressModeU: "repeat",  
  addressModeV: "repeat",  
  magFilter: "nearest",  
  minFilter: "nearest",  
});
```

Sampler fields

The address mode fields describe what should happen if we access a U or V coordinate that is out of bounds (outside the range $[0, 1]$).

“repeat” means that the texture just repeats. So $U = 1.5$ is the same as $U = 0.5$, which is the same as $U = -0.5$, or $U = 1200.5$.

This is very useful. It lets us repeat a texture. Here’s one where instead of 1, we used 4 as the maximum value for U and V with repeat:



Sampler fields (2)

There are two other options for the address mode.

“clamp-to-edge” means that the last texel color on the border gets repeated. Here’s what happens if we make u coordinates clamp, but let v coordinates repeat:

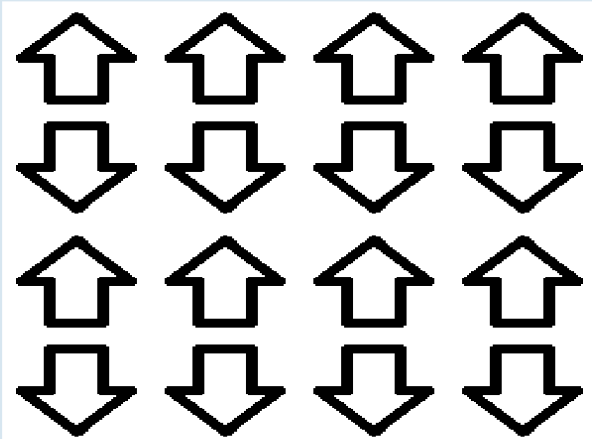


Because we’re clamping the U coordinate, the last column of texels ends up being the color for every U value > 1 .

Sampler fields (3)

Lastly, mirror-repeat means that it will alternate between mirroring and repeating.

I set it to repeat in the U coordinate, but mirror-repeat in the V coord:



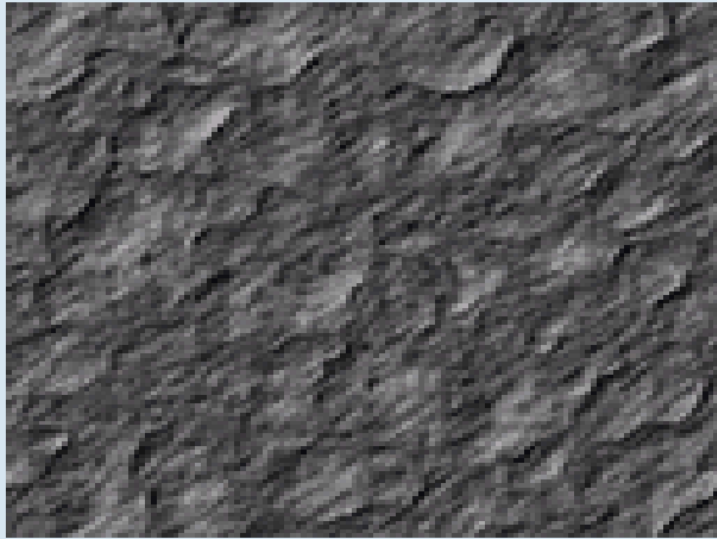
Sampler fields (4)

The other fields make a big difference in image quality:

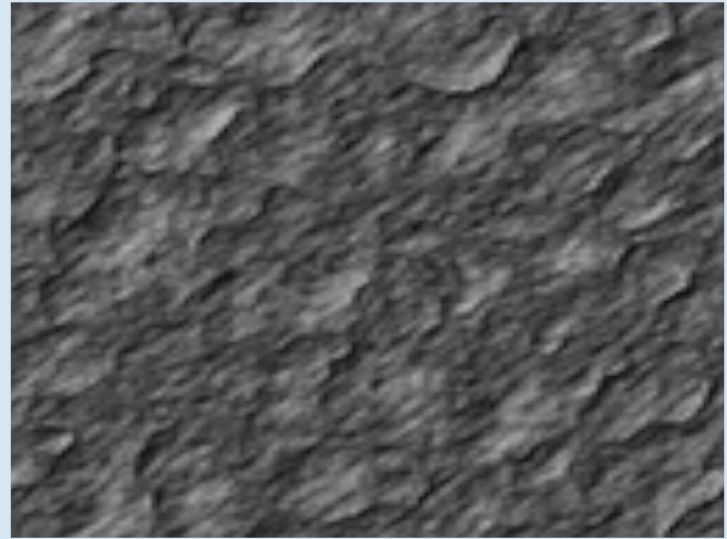
- mag-filter: can be “nearest” or “linear”. This determines what to do when the texture is magnified (i.e., it is spread out over more pixels than there are texels)
- min-filter: can also be “nearest” or “linear”. This determines what to do when the texture is minified (i.e., it is mapped to fewer pixels than there are texels)

We’ll talk more about these in the advanced texturing lecture, but for now, “linear” basically means “smooth”, and “nearest” basically means “grainy.”

Linear vs Nearest comparison



'nearest' filtering



'linear' filtering

Questions?

Next steps

Now that we have a texture and a sampler, we need to load them into a bind group.

Recall that bind groups have layouts which describe the numbers and kinds of data that will be in the bind group.

In our case, that will be a texture at binding 0 and a sampler at binding 1.

Creating bind groups: the layout

Here is the bind group layout for the bind group that will store our texture and sampler:

```
const bindGroup0layout = device.createBindGroupLayout({
  entries: [
    { // tex
      binding: 0, // corresponds to @binding(0)
      visibility: GPUShaderStage.FRAGMENT,
      texture: {} // empty object assigned to `texture` just means
    },           // "this is going to be a texture"
    { // samp
      binding: 1, // corresponds to @binding(1)
      visibility: GPUShaderStage.FRAGMENT,
      sampler: {} // same for samplers
    }
  ]
})
```

Bind group visibility

The `visibility` field determines which shader the variable will be visible to.

It can be `GPUShaderStage.VERTEX`, `GPUShaderStage.FRAGMENT`, or `GPUShaderStage.COMPUTE`, or a bitwise or of any combination of those.

Notice that we aren't loading the texture and sampler here. We're just assigning an empty object to the texture or sampler field, which tells the GPU that there *will* be a texture or sampler there later.

Creating bind groups: the pipeline

Inside our pipeline, we tell it which bind-group layouts we want.

Previously we used `layout: 'auto'`. This is a very limited feature. It will not work correctly if the bind groups are optimized out, and it has other limitations. It's better to define them manually:

```
layout: device.createPipelineLayout({  
    bindGroupLayouts: [  
        bindGroup0layout,  
    ],  
}),
```

Creating bind groups

Now we want to create the bind groups themselves.

```
const bgLayout = pipeline.getBindGroupLayout(0);
const bg = device.createBindGroup({
  layout: bgLayout,
  entries: [
    { // texture
      binding: 0,
      resource: tex.createView(), },
    { // sampler
      binding: 1,
      resource: samp}}]);
```

We can look-up the bind group layout from the pipeline. This is just a convenience.

We can't pass the texture itself: we pass a view. A view is like a region of a texture with colorspace info. `.createView()` just creates a “region” that refers to the whole thing.

Drawing the pass

Finally, we need to draw.

Just like before, we create a command encoder, and we define our render pass, including the pipeline, vertex buffer, and viewport.

But this time, we also set the bind group.

When we draw, the bind group variables in the shader will refer to the values we put inside the bindgroup

Drawing the pass (2)

Assume we called `encoder.beginRenderPass(...)` like before:

```
pass.setViewport(0, 0,  
    context.canvas.width, context.canvas.height, 0, 1);  
pass.setPipeline(pipeline);  
pass.setVertexBuffer(0, vertBuf);  
pass.setBindGroup(0, texBg);  
pass.draw(6);  
pass.end();
```

The only change is setting the bind group before we draw.

Questions?

Giant summary

- Start with a basic quad from a vertex buffer like we had before.
- Add a uv attribute to your vertex data for the texture coordinates.
- Add this attribute to the vertex part of the pipeline.
- Define a bind group layout in your pipeline. It needs a slot for the texture and a slot for the sampler.
- Create a new texture with `device.createTexture(...)`
- Load the texture from a file with `fetch`, `.blob()` and `createImageBitmap()`, or create your own `Uint8ClampedArray` with the RGBA data.
- Use `copyExternalImageDataToTexture` to copy the data from the bitmap or array into the texture memory.

Giant summary (2)

- Create a sampler
- Create the bind group, and load it with the texture and sampler.
- Modify your shader to refer to the texture and sampler.
- Before calling `.draw`, call `.setBindGroup(0, bg)` in your pass.
- Finish the pass and the encoder, and submit to the queue.

Phew. That should draw a texture on screen.

(note, in the sample I'm actually using two bind groups. One for the texture and sampler, and one for a vector that stores the offset. This can help you see another example of a bind group)

This is a lot

Just so you're aware: *I'm* aware that this is a lot. This is a lot to take in.

Our next project is just going to reinforce these same concepts. You'll be loading a texture and drawing some shapes.

The best way to make sure you understand:

- Make sure you can compile and run the sample (sample05_textures)
- Try to start from scratch. When you get stuck, refer to the sample. Then try again. The more you do this, the more the terminology will become concrete (e.g., bind groups, pipelines, layouts, etc.).
- Try loading the arrow texture and make it face different ways. This will help with the project...

Questions?